

AI Newsroom — Execution Plan

This document outlines the end-to-end technical execution roadmap for building an autonomous, agentic Python-based newsroom. The system will ingest daily global events, filter, cluster, verify, write, and automatically publish pocket-sized text news to X (formerly Twitter) and Threads. The publishing window is scheduled for five daily posts between 7:00 PM and 11:00 PM in the configured US timezone.

1. Core Philosophy & Progression Rule

To prevent scope creep and alignment failure, the development lifecycle strictly adheres to the following three-step gate:

1. Stage 0 Audit: Complete a thorough code and architecture evaluation of the existing repository.
2. User Approval Gate: Present the audit findings and obtain explicit sign-off prior to writing any new logic.
3. Incremental Coding: Execute Stages 1 through 11 sequentially with test coverage at every milestone.

2. Stage 0: Repository Audit

An initial technical audit of the codebase at github.com/bilalkarim07/Threads-Automation must be executed prior to any development. The audit will investigate, document, and decide on the preservation or migration of the following components:

- **File Structure & Dependencies:** Catalog all legacy files, python packages, and lockfiles to prevent runtime environment collision.
- **Gemini & LangChain Integrations:** Verify existing model wrappers and assess compatibility with LangChain 1.x / LangGraph 1.x.
- **Threads API Wrapper:** Check stability, rate-limiting handlers, token refresh configurations, and standard payload helpers.
- **Supabase Client:** Assess database connection patterns, safety on connection pooling, and schema migration compatibility.
- **GitHub Actions Setup:** Identify existing workflow triggers, schedule limitations, and secret injection mechanisms.
- **Migration Decisions:** Explicitly document which pieces of the legacy codebase will be retained, refactored, or completely deprecated.

Stage 0 Evaluation Sign-off (Gate 1)

Do not proceed to coding stage 1 until the audit is compiled and signed off below:

Auditor Name

Audit Review Date

--	--

3. News Ingestion Pipeline

The newsroom relies entirely on structured programmatic sources. Direct HTML parsing of social media sites and third-party scraping services (e.g., Apify) are strictly prohibited to ensure high service uptime and legal compliance. Ingestion operates via three isolated phases:

- **Discovery Phase:** Polls GDELT Project, Google News RSS, DuckDuckGo Search (DDGS) API, and validated global RSS feeds (BBC, CNN, ABC, Al Jazeera).
- **Evidence Phase:** Queries official X and Threads search/lookup APIs to extract public reactions, official statements, and real-time community context.
- **Extraction Phase:** Downloads and parses clean core metadata and full text body without tracking scripts or advertisement payloads.

4. Data Engineering & Funnel Mechanics

The pipeline utilizes a structured funnel to filter noise and preserve high informational signal. Raw discoveries pass through the following transformation pipeline: Extract → Normalize → Canonicalize → Hash → Deduplicate → Cluster → Rank → Research → Load.

Stage / Funnel Step	Target Count	Description & Operational Goal
1. Ingested Candidates	25 - 50 Articles	Raw feeds fetched, cleaned of HTML tags, normalized to UTC.
2. Clustered Stories	10 - 20 Clusters	Deduplicated by title-hash and grouped via semantic clustering.
3. Shortlisted Stories	5 - 10 Candidates	Ranked based on source weight, trend velocity, and global relevance.
4. Final Verified News	~5 Stories	Cross-referenced with evidence, fact-checked, and cleared to write.

Database Design & Storage Architecture

The persistent storage layer uses Supabase. Data retention is configured with a sliding 30-day window for ephemeral candidates and lifetime preservation for published content.

Database Table	Primary Purpose	Key Constraints & Indexes
stories	Stores unified global events and core narrative arcs.	
sources	Tracks domain trust scores, credentials, and API rate-limits.	

claims	Isolates verifiable declarative statements for agent review.
evidence	Attaches supporting documentation, source quotes, and metrics.
publications	Logs downstream social media post IDs, timestamps, and engagement.

5. Agentic Workflow (LangGraph 1.x)

Execution states are controlled using a LangGraph 1.x StateGraph. Every node execution must output typed schema states enforced via Pydantic model validations. Execution paths leverage dynamic routing, retries on tool timeouts, and state checkpoints for disaster recovery.

- **Discovery Agent:** Evaluates news feeds, standardizes formats, and populates the candidate database table.
- **Source Intelligence Agent:** Analyzes domain authority, looks up historical citation records, and flags suspicious sources.
- **Research Agent:** Gathers depth context via secondary searches and builds structured evidence logs.
- **Clustering Agent:** Runs vector similarity analyses to combine distinct reports of single events.
- **Selection Agent:** Matches daily publication themes, filters by priority, and selects exactly 5-10 stories.
- **Editorial Agent:** Guides tone consistency, enforces political neutrality, and monitors overall publication flow.
- **Writer Agent:** Crafts compelling, short-form text formats optimized specifically for X and Threads constraints.
- **Fact-check Agent:** Cross-references claims against historical context, database records, and critical redlines.
- **Publisher Agent:** Calls social endpoints, stores post reference IDs, and executes recovery in the event of partial failures.

LLM Fallback & Secret Routing

API keys are injected natively as GitHub Actions Secrets (GEMINI_API_KEY, OPENROUTER_API_KEY, etc.). System variables determine dynamic routing priorities: if Gemini hits quota limits, the router seamlessly falls back to OpenRouter, Groq, or Ollama Cloud according to a configured priority index.

6. Implementation Roadmap

Stage	Phase Name	Key Milestones & Integration Points

Stage 0	Repository Audit	Analyze legacy files, verify dependencies, establish the user review gate.
Stage 1	Foundation	Setup python virtual environment, structure configuration loading.
Stage 2	Source Adapters	Integrate GDELT, Google News RSS, and core international feeds.
Stage 3	Data Engineering	Deploy Supabase schema, configure database indexes and clean routines.
Stage 4	LangGraph Core	Build core StateGraph loop, implement state validation schemas.
Stage 5	Agent Logic	Deploy LLM prompts for specialized roles: Selection, Editorial, Writer.
Stage 6	LLM Router	Configure secondary fallbacks via OpenRouter and local Ollama.
Stage 7	Publishing Adapters	Connect X API and Threads API; configure token retention rules.
Stage 8	Testing Suite	Run unit, integration, mock-agent scenarios, and regression tests.
Stage 9	Dry-Run Execution	Execute 2 full daily cycles to local consoles with publishing mocked out.
Stage 10	Production Live	Activate GitHub Actions scheduler to trigger daily publishing windows.
Stage 11	Future Scope	Introduce AI image generation, analytics dashboards, and trends.

7. Operational Safeguards, Risks & Monitoring

- **Risk: Rate Limiting & API Deprecation.** Action: Multi-layer caching and dynamic fallback intervals are configured in the Publisher Agent.
- **Risk: Hallucinated Claims.** Action: Strict fact-check agent matching against verified sources; hard blocker if similarity to target criteria is below 85%.
- **Rollback Plan:** If publishing error rates exceed 10% in a 24-hour cycle, the GitHub Action automatically disables schedules, shifts to dry-run logs, and opens a critical issue on the repository.

Acceptance and System Sign-off Checklist

- ☐ Stage 0 Audit complete and documented
- ☐ Supabase structural indices and tables configured
- ☐ Gemini fallback pathways and router logic integrated
- ☐ Dry-run validation cycle successfully executed with zero live publishing leaks

Lead Systems Architect Approval

Approval Date